作业1：clip的zero shot classification

In [14]:
```
from PIL import Image
Image.open("./Week10/dog.jpg")
```

Out[14]:

In [13]:
```
# 1、先把模型下载到本地
# modelscope download --model AI-ModelScope/chinese-clip-vit-base-patch16 --loca

import os
import torch
from PIL import Image
from transformers import ChineseCLIPModel, ChineseCLIPProcessor


def clip_zero_shot_classification(image_path: str, candidate_labels: list[str])
    """
    使用 Chinese-CLIP 对本地图片做 zero-shot classification

    :param image_path: 本地图片路径
    :param candidate_labels: 中文候选标签列表
    """
    # 0、检查图片路径
    if not os.path.exists(image_path):
        raise FileNotFoundError(f"图片不存在: {image_path}")

    # 1、读取模型路径
    model_root = os.getenv("MODEL_PATH")
    if model_root is None:
        raise ValueError("环境变量 MODEL_PATH 未设置，请先设置 MODEL_PATH")

    clip_path = os.path.join(model_root, "clip", "chinese-clip-vit-base-patch16"
    if not os.path.exists(clip_path):
        raise FileNotFoundError(f"Chinese-CLIP 模型目录不存在: {clip_path}")
```

```python
    # 2、选择设备
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"[INFO] 当前设备: {device}")
    print(f"[INFO] 模型路径: {clip_path}")

    # 3、加载 Chinese-CLIP
    processor = ChineseCLIPProcessor.from_pretrained(clip_path)
    model = ChineseCLIPModel.from_pretrained(clip_path).to(device)
    model.eval()

    # 4、加载图片
    image = Image.open(image_path).convert("RGB")

    # 5、构造输入
    inputs = processor(
        text=candidate_labels,
        images=image,
        return_tensors="pt",
        padding=True
    )

    # 把输入移动到同一设备
    inputs = {k: v.to(device) for k, v in inputs.items()}

    # 6、模型推理
    with torch.no_grad():
        outputs = model(**inputs)
        logits_per_image = outputs.logits_per_image   # shape: [1, num_labels]
        probs = logits_per_image.softmax(dim=1).cpu().numpy()[0]

    # 7、输出结果
    print("\n=== 分类结果 ===")
    for label, prob in sorted(zip(candidate_labels, probs), key=lambda x: x[1],
        print(f"{label:20s} -> {prob:.4f}")

    best_idx = probs.argmax()
    print("\n=== 最终预测 ===")
    print(f"预测类别: {candidate_labels[best_idx]}")
    print(f"置信度: {probs[best_idx]:.4f}")


if __name__ == "__main__":
    image_path = r"./Week10/dog.jpg"

    # Chinese-CLIP 更建议使用中文标签
    candidate_labels = [
        "一只狗的照片",
        "一只猫的照片",
        "一只鸟的照片",
        "一匹马的照片",
        "一辆汽车的照片",
        "一个人的照片"
    ]

    clip_zero_shot_classification(image_path, candidate_labels)
```

```
[INFO] 当前设备: cpu
[INFO] 模型路径: D:\桌面\typora文件\八斗AI\models\clip\chinese-clip-vit-base-patc
h16
```

```
Loading weights:   0%|          | 0/399 [00:00<?, ?it/s]
ChineseCLIPModel LOAD REPORT from: D:\桌面\typora文件\八斗AI\models\clip\chinese-clip-vit-base-patch16
Key                                  | Status     |  |
-------------------------------------+------------+--+-
vision_model.embeddings.position_ids | UNEXPECTED |  |
text_model.embeddings.position_ids   | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
=== 分类结果 ===
一只狗的照片                -> 0.9947
一只鸟的照片                -> 0.0027
一只猫的照片                -> 0.0013
一匹马的照片                -> 0.0012
一个人的照片                -> 0.0001
一辆汽车的照片               -> 0.0000

=== 最终预测 ===
预测类别: 一只狗的照片
置信度: 0.9947
```

作业2：Qwen VL解析pdf示例

In [2]:

```python
import os
import base64
import pymupdf as fitz
from openai import OpenAI


def pdf_first_page_to_png(pdf_path: str, output_png_path: str, zoom: float = 2.0
    if not os.path.exists(pdf_path):
        raise FileNotFoundError(f"PDF 不存在: {pdf_path}")

    doc = fitz.open(pdf_path)
    if len(doc) == 0:
        doc.close()
        raise ValueError("PDF 没有页面")

    page = doc.load_page(0)
    matrix = fitz.Matrix(zoom, zoom)
    pix = page.get_pixmap(matrix=matrix, alpha=False)
    pix.save(output_png_path)
    doc.close()
    return os.path.abspath(output_png_path)


def image_to_base64_data_url(image_path: str) -> str:
    with open(image_path, "rb") as f:
        image_bytes = f.read()
    b64 = base64.b64encode(image_bytes).decode("utf-8")
    return f"{b64}"


def analyze_pdf_first_page(pdf_path: str):
    api_key = os.getenv("QWEN_API_KEY")
    if not api_key:
```

```python
        raise EnvironmentError("请先设置 QWEN_API_KEY 环境变量")

    png_path = pdf_first_page_to_png(pdf_path, "./first_page.png")
    image_data_url = image_to_base64_data_url(png_path)

    client = OpenAI(
        api_key=api_key,
        base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
    )

    completion = client.chat.completions.create(
        model="qwen3-vl-plus",
        messages=[
            {
                "role": "user",
                "content": [
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": image_data_url
                        }
                    },
                    {
                        "type": "text",
                        "text": (
                            "请解析这页PDF，输出："
                            "1. 标题；"
                            "2. 主要内容摘要；"
                            "3. 关键点列表；"
                            "4. 表格/图表说明；"
                            "5. 最终结论。"
                        )
                    }
                ]
            }
        ]
    )

    print("=== 解析结果 ===")
    print(completion.choices[0].message.content)


if __name__ == "__main__":
    analyze_pdf_first_page(r"D:\桌面\typora文件\八斗AI\第4周：Transformer、Bert和
```

=== 解析结果 ===
根据您提供的PDF页面内容（论文首页），以下是结构化输出：

1. **标题**
   Attention Is All You Need

2. **主要内容摘要**
   本文提出一种全新的神经网络架构——**Transformer**，完全基于注意力机制（attention mechanisms），摒弃了传统的循环（RNN）和卷积（CNN）结构。该模型在机器翻译任务上显著优于当时最优模型：在WMT 2014英-德翻译任务中达到28.4 BLEU（超前2+ BLEU），在英-法任务中达到41.8 BLEU（单模型新SOTA），且仅需3.5天在8块GPU上训练，成本远低于已有模型。此外，Transformer展现出良好的泛化能力，成功应用于英语句法依存分析任务（含大规模与小规模训练数据）。

3. **关键点列表**
   - ✅ **架构创新**：首个完全依赖自注意力（self-attention）与前馈网络的序列建模架构，无递归/卷积。
   - ✅ **核心组件**：
     - 缩放点积注意力（scaled dot-product attention）
     - 多头注意力（multi-head attention）
     - 位置编码（parameter-free positional encoding）
   - ✅ **性能优势**：
     - 英→德翻译：28.4 BLEU（+2+ BLEU over prior SOTA）
     - 英→法翻译：41.8 BLEU（single-model SOTA）
     - 训练高效：3.5天 × 8 GPUs；计算可并行化；训练时间显著缩短
   - ✅ **泛化能力**：成功迁移至英语依存句法分析任务，验证通用性。
   - ✅ **作者贡献说明**（脚注）：
     - Jakob 提出用自注意力替代RNN；
     - Ashish & Illia 设计并实现首个Transformer模型；
     - Noam 提出缩放点积注意力、多头注意力及位置编码；
     - Niki 负责代码库与tensor2tensor实现与调优；
     - Llion 实验新型变体、初始代码库与高效推理/可视化；
     - Łukasz & Aidan 重构代码库，大幅提升效果与效率。

4. **表格/图表说明**
   > 本页为论文首页，**未包含任何表格或图表**。
   > （注：全文后续应有模型架构图、注意力机制示意图、实验结果表格等，但本页仅有文字信息。）

5. **最终结论**
   Transformer 架构证明：**仅靠注意力机制即可构建高性能、高并行化、易扩展的序列建模系统**；它不仅在机器翻译上刷新纪录，还具备强泛化能力，为后续NLP乃至多模态大模型（如BERT、GPT系列）奠定基础——“Attention Is All You Need”成为深度学习发展的重要里程碑。

注：该论文发表于NeurIPS 2017，是Transformer模型的奠基性工作。

In [ ]: